Properties	
name	excel-to-interactive-html-report
description	Use when converting an Excel report to an interactive HTML version via COM automation (Workbooks.SaveAs xlHtml + a DataTables/flatpickr interactivity layer) - especially when building the single-file combined version for email attachment, adding date-range filters, adding a CSV export button, verifying anything on mobile/Safari, building a Dashboard tab (KPIs + Chart.js), adding on-chart value/% labels, a cross-tab drill-down link, a NET/basis-style chart toggle, or an English/Chinese language toggle.

Excel Report -> Interactive HTML Report

Overview

A repeatable two-stage pipeline for turning a periodic Excel report into an interactive HTML version (sortable/searchable/paginated tables, date-range filters, CSV export) without hand-building table markup:

1. **Native export for scaffolding.** Use Excel COM's own "Save As Web Page" to produce the per-sheet `.files` folder and static table markup for free.
2. **Re-skin the sheets that need interactivity.** Keep Excel's native export for sheets that don't need it; replace the specific sheets that do with a DataTables.js + flatpickr template, in the same file-name slot so the rest of the pipeline (tab shell, single-file combiner) doesn't need to know which sheets are native vs. custom.

When to Use

- Rebuilding an Excel-based periodic report (weekly/monthly RPA output) as an HTML version for email/Slack/web viewing
- Adding a date-range filter, per-column dropdown/selection filter, or CSV export button to an Excel-derived HTML table
- A report needs to be attached as ONE file (not a folder) - the single-file combine step below

- Anything about the report "looks fine on desktop but breaks on mobile/iPhone" - see the WebKit gotchas, they're the most common cause
- A sheet's native export includes tables you don't want, or you want a specific labeled sub-table turned into a real interactive DataTables page instead of kept as Excel's native styling - see "Extract a specific sub-table" below

Core Pattern

Stage 1 - native export + cleanup:

```
$wb.SaveAs($HtmlPath, 44)  # 44 = xlHtml
```

Excel's native export embeds an IE-only conditional-comment tab-strip redirect script (`<![if !supportTabStrip]>...window.location.replace(...)...<![endif]>`) that real IE treats as a comment but Chrome/Edge/Firefox execute as plain markup, causing recursive iframe nesting the moment a sheet is opened outside Excel's own frameset. Strip it from every exported sheet with a regex before doing anything else:

```
$pattern = '(?s)<!\[if !supportTabStrip\]>.*?<!\[endif\]>\s*'
```

Stage 2 - re-skin specific sheets. For any sheet needing real interactivity, replace Excel's static `<table>` with a DataTables.js table plus, if needed, a flatpickr date-range toolbar - same output filename, so the shell/combiner logic is unaware of the swap.

Single-file combine (do this in Node, not PowerShell): PowerShell here-strings need every literal `$`/backtick escaped to embed one file's content inside another, which caused real bugs during a manual build. Node's `JSON.stringify` handles all of that safely - the one manual step is neutralizing `</script` sequences in the embedded JSON, since HTML's raw-text tokenizer looks for that exact byte sequence regardless of JS string context:

```
var json = JSON.stringify(sheetsData).replace(/<\/script/gi, '<\\/script');
```

Load each sheet into an `<iframe>` via `frameEl.srcdoc = SHEETS[i].html` on tab click, not `frameEl.src` - this is what makes the result one self-contained file safe to attach to an email, with no separate `.files` folder needed.

Add a client-side "Download CSV" button to any interactive table - exports the CURRENTLY FILTERED rows (respects whatever date-range/column filter is active), built entirely in the browser with a Blob + temporary `<a>`, no server involved:

```
<button type="button" id="download-csv-currency">Download CSV</button>
```

```
function downloadTableCsv(table, filenameBase) {
  function csvEscape(val) {
    val = (val === null || val === undefined) ? '' : String(val);
    val = val.replace(/<[^>]*>/g, '').replace(/ /g, ' ').trim();
    if (/[",\r\n]/.test(val)) { val = '"' + val.replace(/"/g, '""') + '"'; }
    return val;
  }
  var headers = table.columns().header().toArray().map(function (th) { return
$(th).text().trim(); });
  var rows = table.rows({ search: 'applied' }).data().toArray();  // 'applied' =
respects active filters
  var lines = [headers.map(csvEscape).join(',')];
  rows.forEach(function (row) { lines.push(row.map(csvEscape).join(',')); });
  var blob = new Blob(['' + lines.join('\r\n')], { type: 'text/csv;charset=utf-8;'
});  // BOM so Excel doesn't mangle non-ASCII
  var url = URL.createObjectURL(blob);
  var a = document.createElement('a');
  a.href = url;
  a.download = filenameBase + '_' + new Date().toISOString().slice(0,
10).replace(/-/g, '') + '.csv';
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  URL.revokeObjectURL(url);
}
$('#download-csv-currency').on('click', function () { downloadTableCsv(table,
'Currency'); });
```

Duplicate this function per sheet template (each sheet is its own self-contained HTML file/iframe doc in this architecture, so there's no shared module to import from) - just change the button id and `filenameBase` per table. Give the row/table data time to finish loading before wiring the click handler (bind it right after the `DataTable(...)` constructor call returns, inside the same `$(document).ready`, not in a separate script).

Extract a specific sub-table by cell, not by parsing the native export. When a sheet has a labeled sub-region you actually want (e.g. one of two side-by-side tables) and you'd rather drop the rest and get real interactivity for it, skip xlHtml export + HTML-scraping entirely for that region. Locate it robustly by its own text label instead of hardcoding row/column numbers (survives the sheet layout shifting in a future period's file):

```
$labelCell = $ws.Cells.Find("Exact Table Title Text")
$headerRow = $labelCell.Row + 1
$startCol  = $labelCell.Column
# then read header + data rows via .Text (for text/date labels) or .Value2 (for
numbers),
# writing structured {"header":[...],"rows":[...]} JSON exactly like a full-sheet
extraction
```

Feed that JSON into a generic `buildDataTablesHtml(header, rows, title)` builder (the same one used for a plain data-dump sheet) so the sub-table gets full sort/search/CSV-export for free, instead of being stuck with Excel's native look forever.

Give every DataTables instance its own explicit `scrollX`/`autoWidth` decision - don't leave both on by default. See Gotcha 7 below; the short version is `scrollX: true` is for genuinely wide tables that need horizontal scroll, not a safe default to leave on.

Put per-column "selection" dropdown filters in the toolbar, not in a second header row at that column's own position - especially on a wide table. The standard DataTables recipe (a `<select>` per filterable column, in a cloned second `<thead>` row) is correct and works, but on a table with enough columns to need horizontal scroll, a filter for column 20 of 36 renders exactly where you'd expect - which is off to the right, invisible until the user scrolls there. Build the filter `<select>`s in the toolbar instead (same `column(idx).search(...)` wiring, just append the element to a toolbar container instead of a header cell) so every filter stays visible regardless of horizontal scroll position. See Gotcha 9.

Consistent categorical coloring across multiple charts needs ONE fixed key -> color map, not per-chart index-based assignment. If a bar chart and a pie chart both color by the same field (e.g. an entity/DB code) but each independently computes its own list of values-currently-present and assigns colors by array index, the same category can get a DIFFERENT color in each chart - because one chart's list might be alphabetically sorted and the other's sorted by value, or because a filter has changed which values are present in one chart but not the other. Fix: build the color map once, from the full/canonical list of possible values (not just whatever's visible after filtering), and have every chart look up colors from that same map:

```
var COLOR_MAP = {};
ALL_POSSIBLE_VALUES.slice().sort().forEach(function (v, i) { COLOR_MAP[v] =
PALETTE[i % PALETTE.length]; });
```

```
// every chart: backgroundColor: valuesInThisChart.map(function (v) { return
COLOR_MAP[v]; })
```

This keeps colors stable both across charts on the same page AND across filter changes.

Don't gate the HTML build on an already-dated snapshot filename. If another phase of the pipeline (e.g. an email-draft step) needs this HTML to already exist, compute the report's own date stamp directly from the underlying data (e.g. "the Sunday completing the latest populated week") instead of parsing it out of a filename someone else produces - otherwise you have a hidden ordering dependency between phases that don't actually need one.

Dashboard/Chart Extensions (drill-down, on-chart labels, basis toggles, i18n)

These four patterns came out of a from-scratch interactive Dashboard tab (KPIs + Chart.js) built on top of the same iframe-per-sheet architecture, not just plain DataTables pages - reuse them together whenever a report gets a "Dashboard" tab.

Cross-iframe drill-down (click a number on a summary tab -> jump to Data tab, pre-filtered to exactly the transactions behind it). Each sheet is its own `srcdoc` iframe, same-origin with the parent shell, so `window.parent.postMessage` works reliably. Pattern: a summary table's metric cell gets a clickable span whose click handler posts `{type: 'yourAppDrilldown', targetTab: 'Data', filters: {...}}`; the parent shell listens for that message, switches to the named tab, and - once that iframe's `load` event fires - calls the iframe's own `applyExternalFilter(filterMap)` function (parallel to the existing `applyTheme(theme)` cross-iframe contract). `filters` should carry real column names -> values, built from the clicked row's label-column values plus (if the summary table pairs each metric with its own year, e.g. "... 2026"/"... 2025" columns) the year parsed from the clicked column's own header text. When one drill-down category maps to SEVERAL underlying codes in that column (e.g. a "Fittings" link standing for `['HDL-', 'KNOB', 'LHDL', 'SHDL']`), accept an array as the filter value and build a regex alternation - `'^(' + vals.map(regexEscape).join('|') + ')$'` fed through the

column's existing regex-mode `.search()` call - rather than trying to force it through a single-value dropdown-style exact match.

Two distinct, non-obvious DataTables race conditions come with this, both only visible when you actually click through the flow (not from reading the code):

1. The parent's `frameEl.addEventListener('load', ...)` can fire and call the new iframe's `applyExternalFilter` before that iframe's own `$(function(){...})` ready callback has assigned its `table` variable at all - fix: have `applyExternalFilter` check `if (!table) { queue and return; }` instead of calling `table.column(...)` unconditionally.
2. **Subtler**: `initComplete` (the natural place to flush that queued request) itself fires DURING `DataTable()` construction - i.e. BEFORE the outer `table = $(...).DataTable({...})` assignment statement has actually completed - so even the queued-flush-in-initComplete fix silently re-queues forever (no crash, filters just never apply; every dropdown reports "All" and the full unfiltered row count). Fix: assign `table = api;` (the `this.api()` object `initComplete` already receives) as the FIRST statement inside `initComplete`, not just inside the outer chained call.

Simpler alternative that sidesteps both races by construction: have the target tab PULL the pending filter instead of the shell PUSHing it on `load`. Same-origin `srcdoc` iframes give direct synchronous access to `window.parent` (no `postMessage` required at all, though it still works if preferred for other reasons) - so the shell just sets a plain `window.pendingFilter` property before switching tabs, and the target sheet reads `window.parent.pendingFilter` itself as the LAST statement inside its own `$(function(){...})` ready block, right after its `table = $(...).DataTable({...})` call and any `applyExternalFilter` definition. By that point in that sheet's own linear execution, `table` is unconditionally already assigned (the constructor call completed synchronously several statements earlier) - there's no `load`-event race to guard against because nothing is waiting on `load` at all. Confirmed on a 36k-row table: the shell's `load` listener fired ~1.2s before that table's own ready-callback even reached the filter-application code, so a push-based handoff missed every time in that case; the pull-based read never raced because it happens inside the same synchronous flow that finishes initializing the table.

Verify with an exact numeric reconciliation, not just "the tab switched": the sum of the resulting filtered rows' relevant column must equal the value that was clicked.

On-chart data labels ("1.2M (23.4%)" printed directly on a bar/slice, not just on hover) - a from-scratch Chart.js plugin (`Chart.register({id, afterDatasetsDraw})` that reads each dataset's total, computes each element's % share, and draws `fmtCompact(value) + ' (' + pct.toFixed(1) + '%)'` at a per-chart-type anchor point. Every one of the following was found by looking at an actual rendered screenshot, not by reading the plugin code - budget for a real render-and-look pass, every time:

- Horizontal bar: anchor past the bar's own end (`x: element.x + 6, align: 'left'`), not centered inside it - a short bar's centered label collides with the category axis to its left.
- Vertical bar: anchor above the bar top, with axis `suggestedMax: max * 1.2-1.35` for headroom so the tallest bar's label doesn't clip off-canvas.
- Long tail of many adjacent bars: shorten the label text itself (`fmtCompact`: 1.2M/563K, not full digit strings) AND zigzag alternating labels to a second, higher vertical tier - either alone still leaves neighbors' text running together.
- Pie with one dominant slice and several small ones: below a `smallSlicePct` threshold (e.g. 8%), push the label out past the pie's edge along the slice's own mid-angle with a thin leader line back to the slice, staggering leader length by index so several small adjacent slices don't land on the same ring.

A NET/NET NET/ALL-style 3-way basis toggle, independently instantiated per chart (`createBasisToggle(containerId, onChange, initial)` returning `{ get(), relabel() }`) lets each chart on a dashboard show one of two parallel metric bases, or both at once (ALL = a second dataset/series added alongside the first - a grouped bar, a second concentric pie ring, or extra dashed lines on a trend chart - not a separate chart). Keep the three options' INTERNAL value (what `onChange`/comparisons use) as a fixed literal, and put the DISPLAYED button text in a separate attribute (`btn.dataset.value = opt; btn.textContent = <label>`) from day one, even before any i18n requirement exists - see Gotcha 12 below for why conflating the two bites you later.

A language toggle (e.g. English / Traditional Chinese) that translates UI chrome only - titles, headers, labels, button text, chart titles/legends, tab names - and deliberately never touches actual data values or a data table's own column headers. Pattern: one shared `I18N = { en: {...}, zh: {...} }` dict + a `t(key)` lookup helper, embedded into every sheet's iframe template the same way `SHARED_CSS`/a shared JS helper block already is; static translatable elements get a `data-i18n="key"` (and `data-i18n-title="key"` for a tooltip) attribute instead of hardcoded text, and a generic `applyLanguage(lang)` function does

`document.querySelectorAll('[data-i18n]').forEach(el => el.textContent = t(el.dataset.i18n))` plus whatever page-specific re-render is needed for JS-constructed text (chart titles/legend labels, a summary-bar row count string). Wire it into the parent shell exactly like the existing theme toggle: `currentLang` + `localStorage`, and on each iframe's `load` event call its exposed `applyLanguage(currentLang)` (parallel to `applyTheme`). See Gotcha 12 for the one sharp edge.

Draggable per-column resize handles (a `<span class="col-resize-handle">` on each header cell's right edge, mousedown/mousemove/mouseup driven, width persisted per-column to `localStorage`) - ported cleanly from one report's hardcoded-width pivot table to a DIFFERENT report's generic multi-table builder shared across several differently-shaped sheets, this needed three changes beyond a literal copy-paste (all three only surfaced by actually dragging a handle in a real browser, not from reading the ported code):

1. **Requires `table-layout: fixed`, applied only AFTER measuring every column's natural width.** Without `table-layout:fixed`, an explicit `<th>` width is just a hint - the browser's default `auto` layout re-measures from cell content and ignores it, so drag-resize visibly does nothing. But switching to `fixed` before every column has an explicit width makes the browser split the table's space evenly across untouched columns instead, squeezing their header text into an illegible pile-up - invisible in a code review, glaring in an actual screenshot. Fix: on first render, measure each header cell's current (auto-layout) `getBoundingClientRect().width`, THEN switch to `table-layout:fixed` and apply those measured widths (or a saved custom one) explicitly to every column, not just the one being resized. This is also what "Reset column widths" should restore, not a blank width.
2. **Don't ship it on a table that also needs `scrollX: true`.** DataTables' scrollX mode clones the header row into a separate scrolling wrapper and re-derives its own column widths on redraw/`columns.adjust()` - a resize drag applied to the original header node had zero visible effect on a 103-column scrollX table, confirmed only by actually dragging it (the width change landed in local component state/localStorage correctly, so a lazier check reading those back would have reported success). This is the same width-management territory as Gotcha 7 below; scope the feature to non-scrollX tables only rather than half-shipping it.
3. **Fitting a table's total width to the viewport ("Excel Fit-to-Page" for a laptop screen) requires `box-sizing: border-box` on every `<th>`/`<td>`, or the**

computed scale is systematically wrong. Scaling every column's measured natural width by `availableWidth / naturalTotalWidth` and applying the results as `th.style.width` looks correct and internally consistent (the applied `style.width` values do sum to exactly the container width) - but the table still visibly overflowed the screen, because `style.width` under the CSS default `box-sizing: content-box` sets only the content box, and DataTables' own header/cell padding (~15-18px each side, unaccounted for) gets added on top of every single column when actually rendered - `getBoundingClientRect().width` on each cell came out ~30px wider than its own `style.width` said, invisible unless you specifically compare the two rather than just checking that the styles you set sum correctly. Fix: `box-sizing: border-box` on the table's cells, so `style.width` means the actual total rendered width, matching what the fit-to-width math assumes.

4. **Mousedown/mouseup on the resize handle still fires a browser `click` afterward, which bubbles up into DataTables' own sort listener on the `<th>`** - `stopPropagation()` on the handle's `mousedown` does NOT stop that separate `click` event, so every resize drag was silently ALSO toggling that column's sort direction. Confirmed with a plain click (no drag at all) directly on the handle: `table.order()` still flipped. Subtler still: a drag that moves the cursor far enough can end with `mouseup` landing on a NEIGHBORING header cell (the handle moved out from under the cursor as the column widened), so the resulting click can land on a different `<th>` than the one being resized - stopping propagation on that one handle's own `click` listener isn't enough either. Fix: on `mousedown`, register a ONE-SHOT, CAPTURING-phase `click` listener on `document` that calls `stopPropagation()`+`preventDefault()` and immediately removes itself - this swallows the very next click anywhere on the page, regardless of which element it lands on, before any bubble-phase sort listener sees it.

"Total Index" KPI cards + a pie chart, on a plain (non-Dashboard) DataTables page - ported from a different report's Dashboard-style "Total X (weighted %)" KPI pattern onto an otherwise ordinary generic-builder table page (no live multiselect filters of its own). Two things made this port generic instead of another hardcoded-per-sheet block: (1) detect year-paired metric columns automatically by matching header text like `"<Metric> 2025"`/`"<Metric> 2026"` (reusing the same year-regex the metric-column drilldown already parses headers with) rather than naming columns explicitly, so it works unmodified across several differently-shaped sheets sharing this one builder function; (2) since these sheets keep Excel's own formatted display text ("$1,234$", "22.4%"), treat parens as negative) - and ratio/%-named column pairs are excluded from summing entirely, since summing a

percentage across rows isn't a meaningful total. Each KPI's "weighted %" is that year's share of the SAME metric's 2025+2026 combined total, not a share of some cross-metric grand total. The pie groups by the sheet's own first label column, valued by whichever metric pair's header contains "sales" (falling back to the first pair) - reuses the Dashboard's own on-chart label plugin/palette/fmtCompact`, factored out as a second page's own copy of that shared block (not a refactor of the Dashboard's already-verified copy) so it renders identically.

Gotchas (all found by testing, not by reading the code)

1. **flatpickr silently swaps its UI on touch devices.** By default, on a touch device flatpickr sets your input to `type="hidden"` and injects a native `<input type="date">` sibling that exchanges values in ISO format (`YYYY-MM-DD`) regardless of your app's configured `dateFormat` - a completely different, usually untested code path from desktop. Fix: `disableMobile: true` on every flatpickr instantiation, so one calendar UI runs everywhere.

2. **`new Date(string)` is not reliable across JS engines for partial date strings.** `new Date("2026/01")` (year/month, no day) parses leniently as Jan 2026 in Chromium/V8, but returns `Invalid Date` in WebKit/JavaScriptCore - the actual engine behind every iOS browser (Safari, Chrome-on-iOS, everything - Apple requires it). A row whose date silently becomes `null` this way can make an entire filtered table show zero rows on every iPhone/iPad while looking perfect on desktop. Never feed an ambiguous string to `new Date()`; parse it with a regex and construct explicitly: `new Date(year, month - 1, day)`, which is engine-independent.

3. **Chromium + device/viewport emulation is not real mobile testing.** Playwright's `devices['iPhone 13']` under the `chromium` engine only emulates viewport size and touch flags - it's still V8 underneath, so it will NOT catch either gotcha above. Use the real `webkit` engine (`playwright.webkit`, `playwright install webkit`) for any claim of "verified on mobile."

4. **`$PSScriptRoot` is empty during a script's own `param()` default-value evaluation when launched via `powershell -File`**, but resolves correctly when invoked via the `&` call operator from a wrapper script - which is how production

pipelines usually invoke each phase. When dry-running such a script standalone, invoke it the same way production does (`& $scriptPath`), not `-File`, or you'll chase a phantom bug that only exists in your test harness.

5. **CSV export must read the currently filtered rows (`search: 'applied'`), not the whole table** - see the full snippet in Core Pattern above. Getting this wrong means "Download CSV" silently exports the entire dataset even when the user has narrowed the view to one month, which looks like a much smaller bug than it is.

6. **Verify with a real headless-browser pass, not code review alone.** Every gotcha above is invisible from reading the source - they only surface when something actually taps the element, reads back live DOM/JS state (input type, computed row count, calendar open/closed), and checks the result. A quiet default-filter row count of 0 on one specific tab/engine combination is the kind of thing only a real run catches.

7. `scrollX: true` **combined with** `autoWidth: true` **can intermittently miscalculate a narrow table's column widths, visually shifting cell content into the next column - while the underlying data/DOM order stays completely correct.** `scrollX` makes DataTables split the header and body into two separate DOM tables (a cloned header-only table plus a scrolling body table) kept visually aligned via JS-computed pixel widths; on a table whose total column width comfortably fits the container (i.e. doesn't actually need horizontal scroll), this width-sync calculation can come out wrong for a column with short content, making its text spill rightward into the next header's column - looking exactly like "the data shifted one column over" even though `table.rows().data()` / a DOM query on `<td>` text proves every cell is in the right place. This is genuinely rendering-timing-dependent (not deterministic), so a headless Playwright check can pass clean on the very build where a real user's browser shows it - **passing an automated check is not sufficient proof this is fixed** if `scrollX`/`autoWidth` are involved; re-verify by also checking rendered `boundingBox()` positions of header vs. data cells, ideally across several fresh page loads. Fix: don't default `scrollX: true` for every table - only enable it for tables that actually need horizontal scroll (many columns, genuinely wider than the viewport); for everything else use `scrollX: false, autoWidth: false` and let the browser's own (deterministic) table layout size the columns.

8. **A hand-rolled regex-escape helper that loops over a list of special characters, doing `str.split(c).join('\' + c)` for each one, MUST escape**

the backslash character itself FIRST - not last, and not just "somewhere in the list". Escaping any other character (e.g. `.`) inserts a brand-new backslash into the string; if the backslash entry is processed *after* that in the same sequential loop, that pass finds the backslash it just inserted and escapes it too - doubling every backslash that was meant to escape a real character. The resulting string still *looks* plausible (no crash, no console error) but is silently wrong: `new RegExp('^' + escaped + '$')` built from it treats `\\.` as "literal backslash, then any character" instead of "literal period", so an exact-match column filter on any value containing a period (a very common case - company names like "A.R.E." or "B.GROUP") matches **zero** rows. This is invisible in code review and in an automated check that only exercises plain alphanumeric values (e.g. testing a filter against `"MAZ"` or `"MHK"` passes clean) - it only surfaces when you specifically test a value containing one of the characters being escaped. Verify any such helper directly: build a regex from its output and assert it matches the ORIGINAL input string for several probe values that contain periods, parens, and a literal backslash - not just the happy-path values already visible in a first test run.

This exact bug class recurred on a later project, from a different root cause: the loop logic was correct (backslash escaped first), but the escape-CHARACTER-COUNT was wrong at the source - each entry in `specials` and the `join()` replacement both need exactly the right number of literal backslash characters to survive TWO rounds of string-literal reduction (once when the generator's own outer template literal is evaluated into the generated page's source, once more when the browser parses that as a JS string literal): 4 backslash characters in the generator's source produce 1 real backslash at runtime; 8 (one escaping round too many - easy to reach by over-applying the "double it" instinct) silently produce 2, and every special character gets double-escaped instead of single-escaped, with the identical zero-rows-matched symptom as above. This is why a fix to this helper should always be verified by printing the ACTUAL runtime string length/value of one escaped special character (e.g. `regexEscape('.').length` should be 2: one backslash + the period), not just by re-reading the loop logic - a correct loop over a miscounted source string reproduces the exact same bug. Caught here specifically because a newly-added Dashboard chart click-to-drill-down happened to be tested against a vendor name ending in "LTD." - the very first drill-down path in the whole project to be exercised against a period-containing exact-match value.

And a third time, in a brand-new `RegExp` LITERAL (not a hand-rolled escape helper's string data this time) written to detect parenthesized-negative numbers for

a custom DataTables sort-type: `/^\(.*\)$/` written directly in the generator's outer template literal. Same root cause as the `regexEscape` recurrence above (one round of backslash-halving too few - a regex literal needs `\\(` in the source to produce `\(` in the generated page, not `\(`), but a different, more dramatic symptom: with the backslashes stripped, the pattern became `/^(.*)$/` - an unescaped capturing group spanning the WHOLE string, so `.test()` returned `true` for every single value, not just parenthesized ones. Every row was silently treated as "negative", flipping the entire column's sort direction - "high to low" rendered as low to high, with no error and no obviously-wrong single value to spot; only comparing the live rendered order against an independently-recomputed expected order caught it. **The general rule, stated once for all three incidents**: any `\` you type directly into this generator's outer template literal - inside a quoted string OR a bare `/regex/` literal, it doesn't matter which - needs doubling to survive being evaluated once as this file's own template literal before the result is treated as source code again by the browser. Grep for a lone `\(`, `\.`, `\d`, etc. (an odd number of backslashes before the escaped character) anywhere new regex-related code is added to this generator, and verify by printing/testing the ACTUAL generated output's behavior - never by re-reading the authored source, which looks correct at every one of these three sites and was wrong every time.

9. **A per-column filter placed in a second header row, on a wide table with `scrollX` enabled, can pass every automated check while being invisible to a real user - not because of a rendering bug, but because it renders exactly where its column lives.** A filter for column 20 of a 36-column table sits at that column's horizontal position on the page; if the table is wide enough to need `scrollX` at all, that position is off-screen by default, and nothing about `getComputedStyle` (`display: table-row`, `visibility: visible`) or `getBoundingClientRect()` (a real, non-zero, correctly-positioned box) reveals this - both report the element as completely normal, because it genuinely is: it's just off in unscrolled space. A first-pass screenshot can even look like the element is silently missing, when scrolling the table horizontally to that column's position shows it was there all along. Two things follow: (a) a screenshot alone isn't enough either - if it's cropped to (or the viewport just happens to only cover) the first several columns, a working, correctly-positioned element several thousand pixels to the right won't appear in it, which looks identical to "not rendering" unless you specifically scroll to or query the element's own real page position first; (b) the actual fix isn't a CSS patch, it's an information-architecture one - move filters that

need to always be visible into the toolbar (see Core Pattern above), not into the table's own column layout, on any table wide enough to scroll.

10. **A cross-iframe drill-down link's target-tab lookup must match on a stable internal id, never on the currently-displayed (and possibly translated) tab label.** If tab buttons are built from a list of `{name, html}` objects and `postMessage({targetTab: name, ...})` / `SHEETS.findIndex(s => s.name === ...)` both key off that same `name` field, adding a language toggle that later renames the VISIBLE button text must not touch `name` itself - give each sheet a separate stable `key` (used only for `t('tab_' + key)` lookups) and keep `name` as the untranslated, load-bearing id every drilldown message already depends on. Getting this backwards silently breaks every drill-down link the moment the user switches language, with no error - `findIndex` just returns -1 and the tab switch quietly does nothing.

11. `table.draw(false)` **after a language switch is what actually refreshes already-rendered cell tooltips/content that were set at** `createdCell` **time (e.g. a drill-link's** `title` **attribute) - a language switch that only walks** `[data-i18n]` **elements will correctly relabel static chrome but leave any dynamically-created cell content (built once, at row-render time, in the previous language) stale until the table is redrawn.** Cheap to add, easy to forget precisely because the bug is invisible on the very first tab you happen to test (whichever one loads with `table` already `null`/undefined the first time, so the "for the record" check unintentionally passes).

12. **A toggle button group whose displayed label doubles as its own internal comparison key (**`b.textContent === current`**) breaks the instant that label becomes translatable - not with an error, but by silently getting stuck: no button's** `textContent` **matches the (still-English) internal state anymore, so** `className = 'active'` **never gets set on the right one and** `onChange` **fires with a value nothing downstream recognizes.** This is easy to miss because it works perfectly right up until a later, unrelated feature (i18n) is added - by which point the coupling is buried inside a helper that "already works" and isn't the file anyone thinks to re-check. Fix preemptively: give every such button a `dataset.value` holding the real internal value, compare against THAT, and drive `textContent` purely from a separate label lookup - do this even in a report with no i18n plans yet, since retrofitting it after the fact means re-touching every call site that reads the old `textContent`- based comparison.

13. **Any regex literal written directly inside the outer template literal that generates a sheet's `<script>` block needs every backslash doubled, or it silently loses them.** The whole `buildDataTablesHtml`-style function body is itself one big backtick template literal in the generator script; an unrecognized escape sequence inside a template literal (`\d`, `\.`, `\(` - none of these are recognized JS string escapes) is dropped per spec, keeping just the letter/character that followed the backslash - so `/^\(?-?[\d,]+(\.\d+)?%?\)?$/` written straight into that outer template literal comes out the other side as `/^(?-?[d,]+(.d+)?%?)?$/` in the generated page - not merely wrong, but an invalid regex that throws `SyntaxError: Invalid group` the instant the generated page's script runs, breaking every feature defined after it (a DataTables `initComplete` that never fires because the whole script died partway through looks, from the outside, exactly like Gotcha 2 of the drill-down race conditions above - "table stays undefined forever" - even though the actual cause here is a dead script, not a timing race). Fix: double every backslash you want to survive into the final regex (`\\d`, `\\.`, `\\(`), the same rule already used for `/\\s(20\\d{2})$/` elsewhere in this pattern - and verify by printing/`eval`-ing the generated source in isolation (not just running the whole build), since a build script with no Node syntax errors of its own gives zero indication that a STRING it emitted is invalid JS.

Real-world case

Found across a PowerShell rebuild of a weekly multi-currency exchange-rate Excel report converted to a 17-tab interactive HTML version: the IE redirect-script bug (found first, 2026-08-19), the flatpickr mobile-swap bug on three date-filtered tabs (found via a real WebKit mobile-emulation session, fixed with `disableMobile: true`), and the WebKit `new Date()` partial-string bug on one specific tab's date filter (found the same way, one day later, while re-verifying the other three tabs after adding a CSV export button) - three independent, silent, desktop-invisible bugs in one interactivity layer, none caught by code review.

A separate monthly Global Purchase Report rebuild (2026-08-25) hit two more of this skill's failure modes on the same interactivity layer: (1) after switching a set of item-level tables from a forced `width:100%` to content-based ("autofit") column sizing, `scrollX` (left on from an earlier iteration) started intermittently spilling the ITEM_CODE column's text into the next column - repeated automated Playwright checks kept coming back clean, yet the user's real browser reproduced it consistently, until

`scrollX`/`autoWidth` were disabled for that table (Gotcha 7); (2) a bar chart and a pie chart both coloring by the same "DB" entity field showed different colors for the same entity, because each computed its own differently-sorted list of visible values and assigned palette colors by index - fixed with one fixed value->color map built from the full entity list. Separately, the project's Exchange Rate sheet was rebuilt using the "extract a specific sub-table by cell" pattern above: instead of keeping Excel's native export (which included an unwanted second currency table right next to the wanted one), the wanted table was located via `Cells.Find()` on its own title text and its header + data rows read directly via COM, then rendered through the same generic DataTables page builder already used for a plain data-dump sheet - turning a static native-Excel-styled table into a fully sortable/searchable/CSV-exportable one with no HTML-scraping involved.

The same project (2026-08-26) added per-column dropdown "selection" filters plus a "clear all filters" button to the 44,000-row main Data tab, scoped to five high-value columns (Vendor Name, DB, Commodity, Month, Ignore/Consider) rather than all 36 - and hit two independent bugs building it, one after the other. First, the initial version (a second `<thead>` row of `<select>` elements, `orderCellsTop: true` so sort clicks still target the real header row) is exactly where Gotcha 8's regex-escape bug was found: the feature looked complete and a first Playwright pass (filtering by `DB="MAZ"`) came back clean, only breaking once a vendor name containing periods was specifically tested. Second, after fixing that, a real screenshot showed the filter row simply wasn't there - despite `getComputedStyle`/`getBoundingClientRect()` both reporting it as a normal, visible, correctly-sized element. The actual cause was Gotcha 9: those five filter columns sit at indices 19-24 of 36, and this table needs `scrollX` - so the filter row was rendering exactly where it should, several thousand pixels to the right of the default viewport, not broken at all. The fix was architectural, not a CSS patch: move the five `<select>`s into the toolbar (always visible, independent of horizontal scroll position) instead of a second header row tied to each filter's own column position - confirmed by an actual screenshot of the toolbar showing all five dropdowns, not by any DOM query, since DOM queries had already given a false sense of confidence once.

A monthly MTC Sales & Margin report rebuild (2026-08 - the source project for the whole "Dashboard/Chart Extensions" section above) added a full Dashboard tab and then iterated it across five separate feature requests, each layered on the last: an 8-KPI NET/NET NET pair, a Country-of-Origin pie, a per-chart NET/NET NET/ALL toggle, a cross-tab drill-down from five different YoY comparison tabs (plus a sixth, structurally different Customer-Quarter tab using a fixed page-filter year instead of paired year

columns) into the main Data tab, and finally an English/Traditional-Chinese language toggle. Every one of Gotchas 10-12 was hit in exactly this build, in this order: the basis-toggle's `textContent`-as-comparison-key coupling (12) only became a real bug once the language toggle was added weeks after the toggle itself, at which point every button silently stopped tracking its own active state; the drill-down's `name`-as-tab-id (10) was caught before shipping only because the language toggle's own smoke test happened to click a drill-link right after switching languages; and the stale-tooltip issue (11) was caught the same way, by re-running the *original* drill-down verification script (not a new one written for i18n) after the language switch, confirming its tooltip text had actually updated. The general lesson: a new cross-cutting feature (i18n here) doesn't just need its own tests - re-running each EARLIER feature's existing verification, now with the new feature also active, is what actually surfaces this class of bug.